

Task 1 — Load, explore, and preprocess

We inspect dimensions, missing values, duplicates, descriptive statistics, and feature distributions. If `Id` exists, it is dropped because it is an identifier rather than a measured input. Missing input values are imputed using the training-set median. The continuous `quality` target is retained for Linear Regression; a binary target is derived for Logistic Regression.

```
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import VarianceThreshold, SequentialFeatureSelector
from sklearn.decomposition import FactorAnalysis
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score,
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, ConfusionMatrixDisplay
)
```

```
df = pd.read_csv("wineQ1.csv")
print("Dataset shape:", df.shape)
display(df.head())
display(df.describe().T)
print("\nMissing values:\n", df.isnull().sum())
print("\nduplicate rows:", df.duplicated().sum())
if "Id" in df.columns:
    df = df.drop(columns=["Id"])
assert "quality" in df.columns, "Expected a target column named 'quality'."
print("Shape after dropping Id if present:", df.shape)
print("Input feature count:", df.shape[1] - 1)
```

Dataset shape: (1143, 13)

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	Id
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	0
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5	1
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5	2
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6	3
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5	4

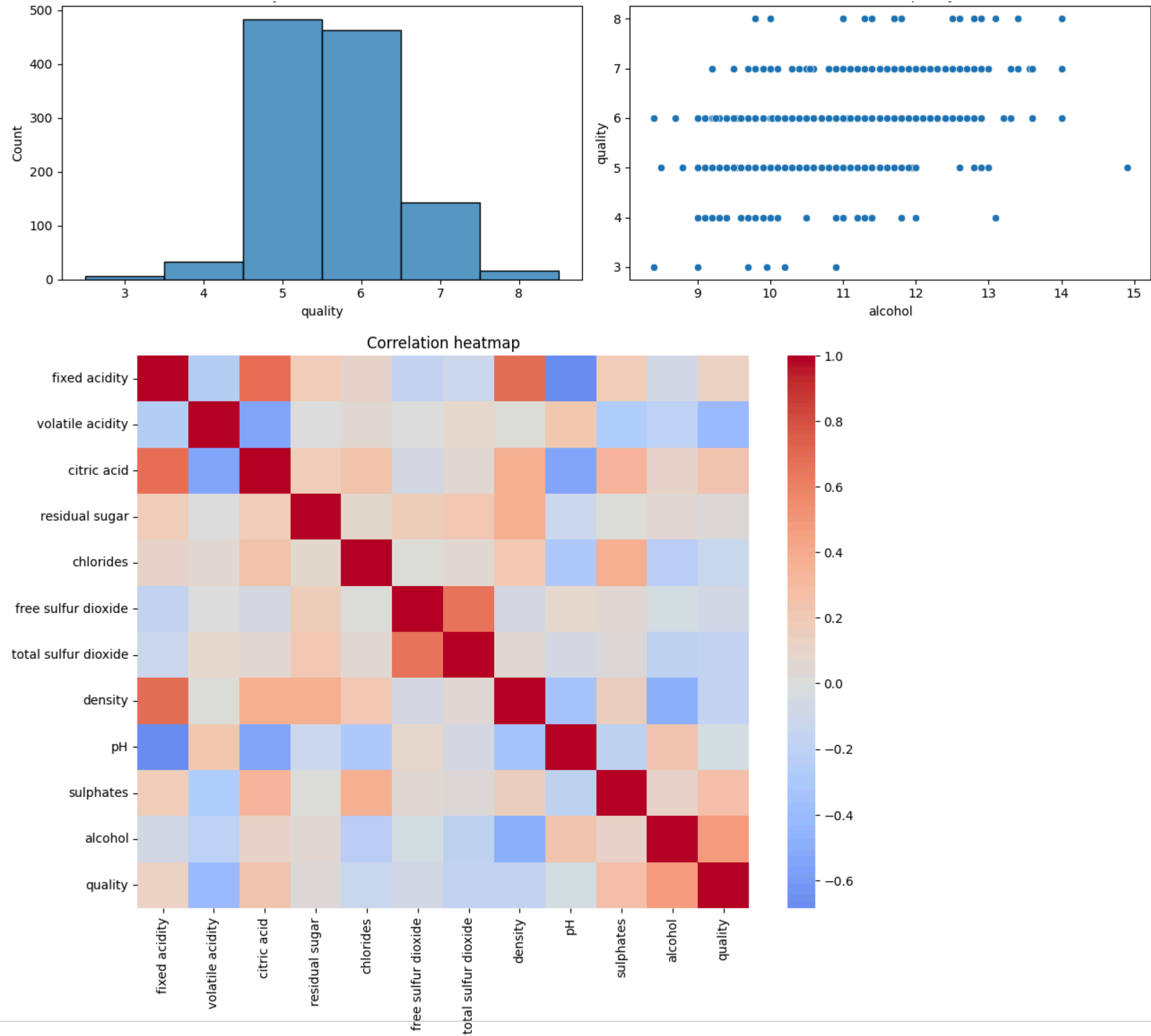
	count	mean	std	min	25%	50%	75%	max
fixed acidity	1143.0	8.311111	1.747595	4.60000	7.10000	7.90000	9.10000	15.90000
volatile acidity	1143.0	0.531339	0.179633	0.12000	0.39250	0.52000	0.64000	1.58000
citric acid	1143.0	0.268364	0.196686	0.00000	0.09000	0.25000	0.42000	1.00000
residual sugar	1143.0	2.532152	1.355917	0.90000	1.90000	2.20000	2.60000	15.50000
chlorides	1143.0	0.086933	0.047267	0.01200	0.07000	0.07900	0.09000	0.61100
free sulfur dioxide	1143.0	15.615486	10.250486	1.00000	7.00000	13.00000	21.00000	68.00000
total sulfur dioxide	1143.0	45.914698	32.782130	6.00000	21.00000	37.00000	61.00000	289.00000
density	1143.0	0.996730	0.001925	0.99007	0.99557	0.99668	0.997845	1.00369
pH	1143.0	3.311015	0.156864	2.74000	3.20500	3.31000	3.40000	4.01000
sulphates	1143.0	0.657708	0.170399	0.33000	0.55000	0.62000	0.73000	2.00000
alcohol	1143.0	10.442111	1.082196	8.40000	9.50000	10.20000	11.10000	14.90000
quality	1143.0	5.657043	0.805824	3.00000	5.00000	6.00000	6.00000	8.00000
Id	1143.0	804.969379	463.997116	0.00000	411.00000	794.00000	1209.50000	1597.00000

Missing values:
fixed acidity 0
volatile acidity 0
citric acid 0
residual sugar 0
chlorides 0
free sulfur dioxide 0
total sulfur dioxide 0
density 0
pH 0
sulphates 0
alcohol 0
quality 0
Id 0
dtype: int64

Duplicate rows: 0
Shape after dropping Id if present: (1143, 12)
Input feature count: 11

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
sns.histplot(df["quality"], discrete=True, ax=axes[0])
axes[0].set_title("Quality score distribution")
sns.scatterplot(data=df, x="alcohol", y="quality", ax=axes[1])
axes[1].set_title("Alcohol vs quality")
plt.tight_layout()
plt.show()

plt.figure(figsize=(11, 8))
sns.heatmap(df.corr(numeric_only=True), cmap="coolwarm", center=0)
plt.title("Correlation heatmap")
plt.show()
```



Task 1 — Split and scale without data leakage

The classification target is created with a configurable threshold (default `quality >= 6`). The same split is used for both tasks. The imputer and scaler are fit on training data only, then applied to test data.

```
QUALITY_THRESHOLD = 6
X = df.drop(columns=["quality"]).apply(pd.to_numeric, errors="coerce")
y_reg = pd.to_numeric(df["quality"], errors="coerce")
valid = y_reg.notna()
X, y_reg = X.loc[valid], y_reg.loc[valid]
y_cls = (y_reg >= QUALITY_THRESHOLD).astype(int)

X_train, X_test, yreg_train, yreg_test, ycls_train, ycls_test = train_test_split(
    X, y_reg, y_cls, test_size=0.20, random_state=42, stratify=y_cls
)

imputer = SimpleImputer(strategy="median")
X_train_imp = pd.DataFrame(imputer.fit_transform(X_train), columns=X.columns, index=X_train.index)
X_test_imp = pd.DataFrame(imputer.transform(X_test), columns=X.columns, index=X_test.index)

scaler = StandardScaler()
X_train_scaled = pd.DataFrame(scaler.fit_transform(X_train_imp), columns=X.columns, index=X_train.index)
X_test_scaled = pd.DataFrame(scaler.transform(X_test_imp), columns=X.columns, index=X_test.index)

print("Instances:", len(df))
print("Input features:", X.shape[1])
print("Train instances:", len(X_train), "| Test instances:", len(X_test))
print("Class counts (0=Low, 1=Good):")
display(y_cls.value_counts().rename_axis("class").to_frame("count"))
```

Instances: 1143
Input features: 11
Train instances: 914 | Test instances: 229
Class counts (0=Low, 1=Good):

	count
class	
1	621
0	522

Task 2A — Low Variance Filter

Variance filtering removes features whose variance is below a threshold. The filter is fit on the unscaled training data because standard scaling would make most non-constant feature variances similar. With threshold 0, only constant features are removed.

```
variance_threshold = 0.0
variance_selector = VarianceThreshold(threshold=variance_threshold)
variance_selector.fit(X_train_imp)
low_variance_features = list(X_train_imp.columns[variance_selector.get_support()])
print("Features retained:", len(low_variance_features))
print(low_variance_features)
```

Features retained: 11
['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'ph', 'sulphates', 'alcohol']

Task 2B — High Correlation Filter

If two input features have absolute correlation above the chosen cutoff, one is removed to reduce redundancy. This filter considers feature-to-feature correlation, not correlation with the target.

```
CORR_THRESHOLD = 0.90
corr_matrix = X_train_scaled.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
high_corr_drop = [col for col in upper.columns if any(upper[col] > CORR_THRESHOLD)]
high_corr_features = [col for col in X_train_scaled.columns if col not in high_corr_drop]
print("Removed due to high correlation:", high_corr_drop)
print("Retained features:", high_corr_features)
```

Removed due to high correlation: []
Retained features: ['fixed acidity', 'volatile acidity', 'citric acid', 'residual sugar', 'chlorides', 'free sulfur dioxide', 'total sulfur dioxide', 'density', 'ph', 'sulphates', 'alcohol']

Task 2C — Factor Analysis

Factor Analysis is a feature-reduction technique. It transforms the original measurements into a smaller number of latent factors. These components are not original feature names, so we report them as `Factor_1`, `Factor_2`, etc. The factor model is fit only on the training data.

```
n_factors = min(5, X_train_scaled.shape[1])
factor_model = FactorAnalysis(n_components=n_factors, random_state=42)
X_train_factors = factor_model.fit_transform(X_train_scaled)
X_test_factors = factor_model.transform(X_test_scaled)
factor_names = [f"Factor_{i+1}" for i in range(n_factors)]
X_train_fa = pd.DataFrame(X_train_factors, columns=factor_names, index=X_train.index)
X_test_fa = pd.DataFrame(X_test_factors, columns=factor_names, index=X_test.index)
print("Reduced components:", factor_names)
print("Original feature count:", X_train_scaled.shape[1], " | Factor count:", len(factor_names))
```

Reduced components: ['Factor_1', 'Factor_2', 'Factor_3', 'Factor_4', 'Factor_5']
Original feature count: 11 | Factor count: 5

Task 2D and 2E — Backward Feature Elimination and Forward Feature Selection

Sequential Feature Selector evaluates subsets using cross-validation. We use up to five selected features (or one fewer than the total if there are very few features). Since selection depends on the target, separate subsets are created for regression and classification.

```
n_select = min(5, max(1, X_train_scaled.shape[1] - 1))

backward_reg_selector = SequentialFeatureSelector(
    LinearRegression(), n_features_to_select=n_select, direction="backward",
    scoring="neg_mean_squared_error", cv=3, n_jobs=-1
)
backward_reg_selector.fit(X_train_scaled, yreg_train)
backward_reg_features = list(X_train_scaled.columns[backward_reg_selector.get_support()])

forward_reg_selector = SequentialFeatureSelector(
    LinearRegression(), n_features_to_select=n_select, direction="forward",
    scoring="neg_mean_squared_error", cv=3, n_jobs=-1
)
forward_reg_selector.fit(X_train_scaled, yreg_train)
forward_reg_features = list(X_train_scaled.columns[forward_reg_selector.get_support()])

backward_cls_selector = SequentialFeatureSelector(
    LogisticRegression(max_iter=3000), n_features_to_select=n_select, direction="backward",
    scoring="accuracy", cv=3, n_jobs=-1
)
backward_cls_selector.fit(X_train_scaled, ycls_train)
backward_cls_features = list(X_train_scaled.columns[backward_cls_selector.get_support()])

forward_cls_selector = SequentialFeatureSelector(
    LogisticRegression(max_iter=3000), n_features_to_select=n_select, direction="forward",
    scoring="accuracy", cv=3, n_jobs=-1
)
forward_cls_selector.fit(X_train_scaled, ycls_train)
forward_cls_features = list(X_train_scaled.columns[forward_cls_selector.get_support()])

selection_summary = pd.DataFrame([
    ["Original", X.shape[1], list(X.columns)],
    ["Low Variance", len(low_variance_features), low_variance_features],
    ["High Correlation", len(high_corr_features), high_corr_features],
    ["Factor Analysis", len(factor_names), factor_names],
    ["Backward (Regression)", len(backward_reg_features), backward_reg_features],
    ["Forward (Regression)", len(forward_reg_features), forward_reg_features],
    ["Backward (Classification)", len(backward_cls_features), backward_cls_features],
    ["Forward (Classification)", len(forward_cls_features), forward_cls_features],
], columns=["Method", "Number retained", "Features/components"])
display(selection_summary)
```

	Method	Number retained	Features/components	
0	Original	11	[fixed acidity, volatile acidity, citric acid...	
1	Low Variance	11	[fixed acidity, volatile acidity, citric acid...	
2	High Correlation	11	[fixed acidity, volatile acidity, citric acid...	
3	Factor Analysis	5	[Factor_1, Factor_2, Factor_3, Factor_4, Facto...	
4	Backward (Regression)	5	[volatile acidity, chlorides, total sulfur dio...	
5	Forward (Regression)	5	[volatile acidity, chlorides, total sulfur dio...	
6	Backward (Classification)	5	[volatile acidity, free sulfur dioxide, total ...	
7	Forward (Classification)	5	[volatile acidity, residual sugar, chlorides, ...	

Next steps: [Generate code with selection_summary](#) [New interactive sheet](#)

Task 3 — Linear Regression and Logistic Regression

We compare models trained on the original features and each selected feature set. Regression metrics are MAE, MSE, RMSE, and R^2 . Classification metrics are accuracy, precision, recall, and F1. The binary classification target is derived from the quality score threshold.

```
def regression_metrics(y_true, pred):
    mse = mean_squared_error(y_true, pred)
    return {
        "MAE": mean_absolute_error(y_true, pred),
        "MSE": mse,
        "RMSE": np.sqrt(mse),
        "R2": r2_score(y_true, pred)
    }

def classification_metrics(y_true, pred):
    return {
        "Accuracy": accuracy_score(y_true, pred),
        "Precision": precision_score(y_true, pred, zero_division=0),
        "Recall": recall_score(y_true, pred, zero_division=0),
        "F1": f1_score(y_true, pred, zero_division=0)
    }

reg_feature_sets = {
    "Original": list(X.columns),
    "Low Variance": low_variance_features,
    "High Correlation": high_corr_features,
    "Backward Selection": backward_reg_features,
    "Forward Selection": forward_reg_features,
}

cls_feature_sets = {
    "Original": list(X.columns),
    "Low Variance": low_variance_features,
    "High Correlation": high_corr_features,
    "Backward Selection": backward_cls_features,
    "Forward Selection": forward_cls_features,
}

results = []
for name, cols in reg_feature_sets.items():
    if not cols:
        continue
    model = LinearRegression()
    model.fit(X_train_scaled[cols], yreg_train)
    pred = model.predict(X_test_scaled[cols])
    cv_scores = cross_val_score(LinearRegression(), X_train_scaled[cols], yreg_train, cv=5, scoring="r2")
    results.append({
        "Task": "Regression", "Feature set": name, "Feature count": len(cols),
        **regression_metrics(yreg_test, pred),
        "CV mean": cv_scores.mean(), "CV std": cv_scores.std()
    })

for name, cols in cls_feature_sets.items():
    if not cols:
        continue
    model = LogisticRegression(max_iter=3000)
    model.fit(X_train_scaled[cols], ycls_train)
    pred = model.predict(X_test_scaled[cols])
    cv_scores = cross_val_score(LogisticRegression(max_iter=3000), X_train_scaled[cols], ycls_train, cv=5, scoring="accuracy")
    results.append({
        "Task": "Classification", "Feature set": name, "Feature count": len(cols),
        **classification_metrics(ycls_test, pred),
        "CV mean": cv_scores.mean(), "CV std": cv_scores.std()
    })

lin_fa = LinearRegression().fit(X_train_fa, yreg_train)
pred_fa_reg = lin_fa.predict(X_test_fa)
cv_fa_reg = cross_val_score(LinearRegression(), X_train_factors, yreg_train, cv=5, scoring="r2")
results.append({
    "Task": "Regression", "Feature set": "Factor Analysis", "Feature count": len(factor_names),
    **regression_metrics(yreg_test, pred_fa_reg), "CV mean": cv_fa_reg.mean(), "CV std": cv_fa_reg.std()
})

log_fa = LogisticRegression(max_iter=3000).fit(X_train_fa, ycls_train)
pred_fa_cls = log_fa.predict(X_test_fa)
cv_fa_cls = cross_val_score(LogisticRegression(max_iter=3000), X_train_factors, ycls_train, cv=5, scoring="accuracy")
results.append({
    "Task": "Classification", "Feature set": "Factor Analysis", "Feature count": len(factor_names),
    **classification_metrics(ycls_test, pred_fa_cls), "CV mean": cv_fa_cls.mean(), "CV std": cv_fa_cls.std()
})

results_df = pd.DataFrame(results)
display(results_df.sort_values(["Task", "CV mean"], ascending=[True, False]))
```

	Task	Feature set	Feature count	MAE	MSE	RMSE	R2	CV mean	CV std	Accuracy	Precision	Recall	F1
8	Classification	Backward Selection	5	NaN	NaN	NaN	NaN	0.761448	0.037124	0.768559	0.798319	0.766129	0.781893
5	Classification	Original	11	NaN	NaN	NaN	NaN	0.750525	0.026217	0.777293	0.801653	0.782258	0.791837
6	Classification	Low Variance	11	NaN	NaN	NaN	NaN	0.750525	0.026217	0.777293	0.801653	0.782258	0.791837
7	Classification	High Correlation	11	NaN	NaN	NaN	NaN	0.750525	0.026217	0.777293	0.801653	0.782258	0.791837
9	Classification	Forward Selection	5	NaN	NaN	NaN	NaN	0.738480	0.029110	0.768559	0.803419	0.758065	0.780083
11	Classification	Factor Analysis	5	NaN	NaN	NaN	NaN	0.716532	0.057986	0.742358	0.768595	0.750000	0.759184
3	Regression	Backward Selection	5	0.503251	0.431339	0.656764	0.378712	0.349696	0.065464	NaN	NaN	NaN	NaN
4	Regression	Forward Selection	5	0.503251	0.431339	0.656764	0.378712	0.349696	0.065464	NaN	NaN	NaN	NaN
0	Regression	Original	11	0.497891	0.422619	0.650092	0.391272	0.343928	0.072757	NaN	NaN	NaN	NaN
1	Regression	Low Variance	11	0.497891	0.422619	0.650092	0.391272	0.343928	0.072757	NaN	NaN	NaN	NaN
2	Regression	High Correlation	11	0.497891	0.422619	0.650092	0.391272	0.343928	0.072757	NaN	NaN	NaN	NaN
10	Regression	Factor Analysis	5	0.539159	0.482765	0.694813	0.304639	0.287771	0.093852	NaN	NaN	NaN	NaN

Task 3 — Gradient Descent: learning rate and iterations

The following cells implement batch Gradient Descent manually for Linear Regression and Logistic Regression. Inputs are standardized, and the target for the linear demonstration is normalized for stable optimization. Loss values are recorded during training so that convergence can be visualized. Try different learning rates and iteration counts.

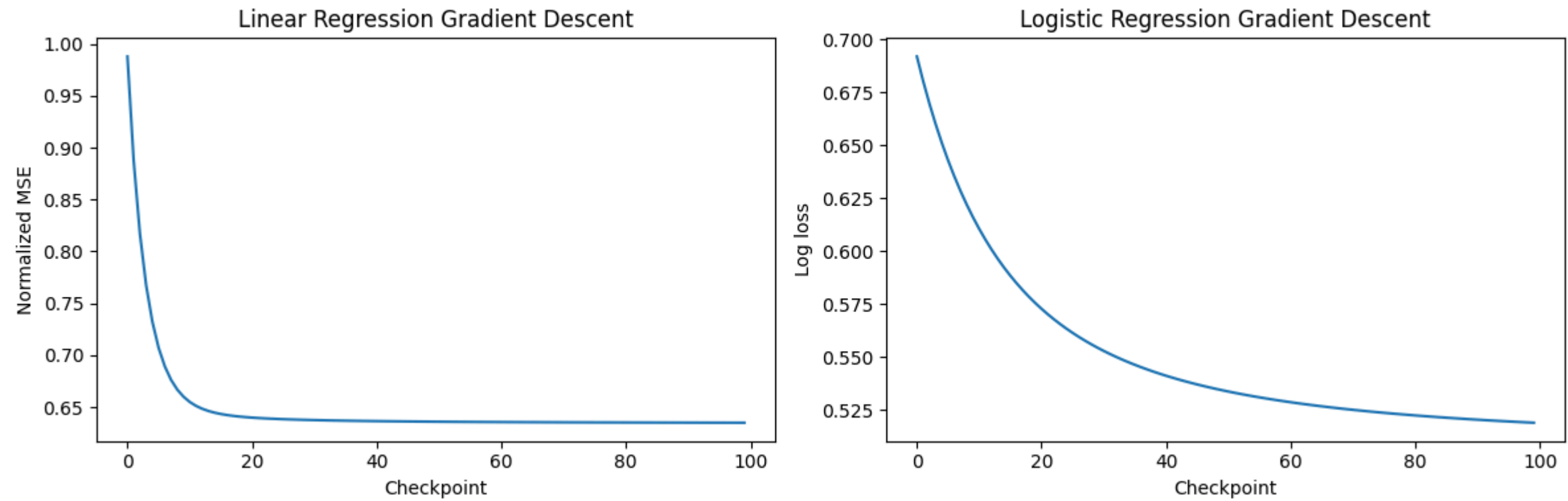
```
def batch_gradient_descent_linear(X_array, y_array, learning_rate=0.01, iterations=1000):
    X_array = np.asarray(X_array, dtype=float)
```

```
y_array = np.asarray(y_array, dtype=float)
x_mean, x_std = X_array.mean(axis=0), X_array.std(axis=0)
x_std[x_std == 0] = 1
Xs = (X_array - x_mean) / x_std
y_mean, y_std = y_array.mean(), (y_array.std() or 1)
ys = (y_array - y_mean) / y_std
Xb = np.c_[np.ones(len(Xs)), Xs]
weights = np.zeros(Xb.shape[1])
losses = []
for i in range(iterations):
    errors = Xb @ weights - ys
    weights -= learning_rate * (Xb.T @ errors) / len(ys)
    if i % max(1, iterations // 100) == 0:
        losses.append(np.mean((Xb @ weights - ys) ** 2))
return weights, losses

def batch_gradient_descent_logistic(X_array, y_array, learning_rate=0.01, iterations=1000):
    X_array = np.asarray(X_array, dtype=float)
    y_array = np.asarray(y_array, dtype=float)
    x_mean, x_std = X_array.mean(axis=0), X_array.std(axis=0)
    x_std[x_std == 0] = 1
    Xs = (X_array - x_mean) / x_std
    Xb = np.c_[np.ones(len(Xs)), Xs]
    weights = np.zeros(Xb.shape[1])
    losses = []
    for i in range(iterations):
        probabilities = 1 / (1 + np.exp(-np.clip(Xb @ weights, -30, 30)))
        weights -= learning_rate * (Xb.T @ (probabilities - y_array)) / len(y_array)
        if i % max(1, iterations // 100) == 0:
            probabilities = np.clip(1 / (1 + np.exp(-np.clip(Xb @ weights, -30, 30))), 1e-9, 1-1e-9)
            loss = -np.mean(y_array*np.log(probabilities) + (1-y_array)*np.log(1-probabilities))
            losses.append(loss)
    return weights, losses

LEARNING_RATE = 0.01
ITERATIONS = 1000
_, linear_losses = batch_gradient_descent_linear(X_train_scaled, yreg_train, LEARNING_RATE, ITERATIONS)
_, logistic_losses = batch_gradient_descent_logistic(X_train_scaled, ycls_train, LEARNING_RATE, ITERATIONS)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].plot(linear_losses)
axes[0].set_title("Linear Regression Gradient Descent")
axes[0].set_xlabel("Checkpoint")
axes[0].set_ylabel("Normalized MSE")
axes[1].plot(logistic_losses)
axes[1].set_title("Logistic Regression Gradient Descent")
axes[1].set_xlabel("Checkpoint")
axes[1].set_ylabel("Log loss")
plt.tight_layout()
plt.show()
```



Experiment: compare learning rates and iterations

A steadily decreasing loss suggests convergence. If loss grows or fluctuates, reduce the learning rate. Increasing iterations gives the optimizer more updates, but it does not guarantee better test performance.

```
experiment_rows = []
for lr in [0.001, 0.01, 0.05, 0.1]:
    for iters in [100, 500, 1000]:
        _, loss = batch_gradient_descent_linear(X_train_scaled, yreg_train, lr, iters)
        experiment_rows.append({
            "Learning rate": lr,
            "Iterations": iters,
            "Final normalized MSE": loss[-1]
        })
gd_experiments = pd.DataFrame(experiment_rows)
display(gd_experiments.sort_values("Final normalized MSE"))
```

	Learning rate	Iterations	Final normalized MSE	
11	0.100	1000	0.634292	
10	0.100	500	0.634298	
8	0.050	1000	0.634298	
7	0.050	500	0.634398	
9	0.100	100	0.634955	
5	0.010	1000	0.634968	
6	0.050	100	0.635960	
4	0.010	500	0.635987	
3	0.010	100	0.655422	
2	0.001	1000	0.656137	
1	0.001	500	0.711106	
0	0.001	100	0.890865	

Task 4 — K-fold Cross-Validation

We report the mean and standard deviation of validation scores. For regression we use R^2 ; for classification we use accuracy. A higher mean is generally better for these metrics, while a lower standard deviation suggests less variation across folds. Cross-validation is performed on training data only.

Task 5 — GridSearchCV

GridSearchCV tunes Ridge Regression's α and Logistic Regression's C . The best parameters and cross-validation score are reported, and the tuned models are evaluated on the held-out test set. The test set is not used to choose hyperparameters.

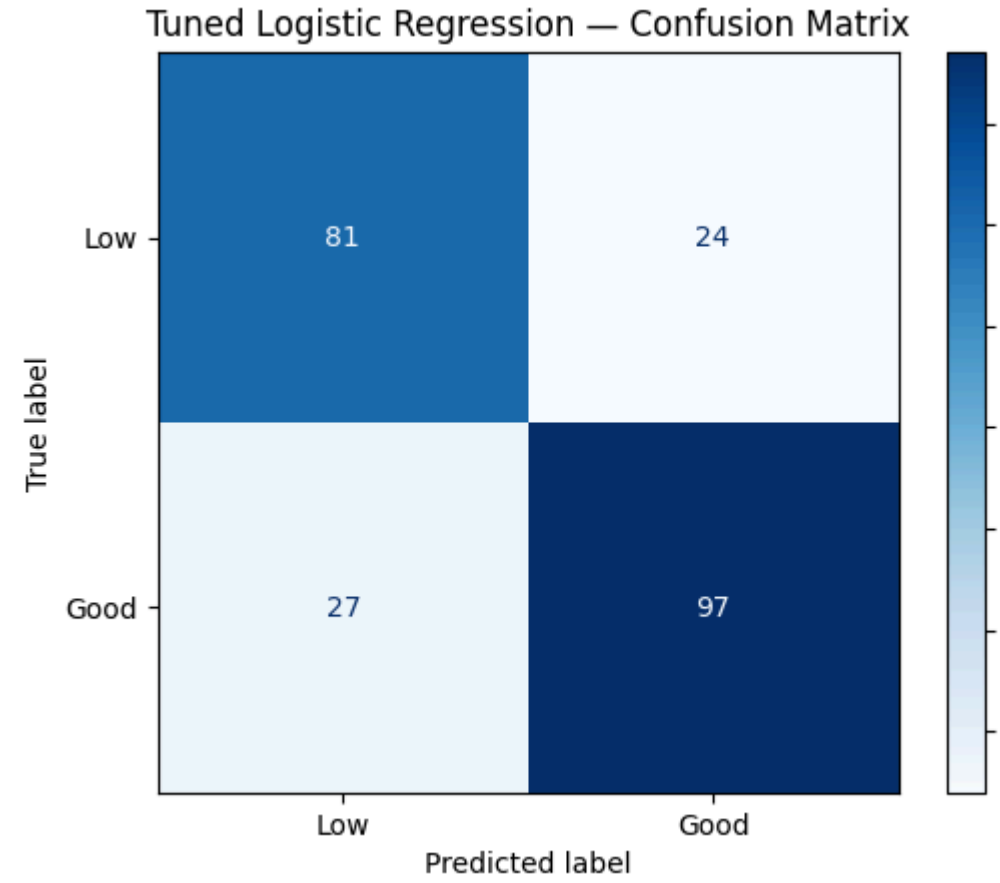
```
ridge_grid = GridSearchCV(
    Ridge(),
    param_grid={"alpha": [0.01, 0.1, 1, 10, 100]},
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=-1
)
ridge_grid.fit(X_train_scaled, yreg_train)
ridge_pred = ridge_grid.best_estimator_.predict(X_test_scaled)

logistic_grid = GridSearchCV(
    LogisticRegression(max_iter=3000),
    param_grid={"C": [0.01, 0.1, 1, 10, 100]},
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)
logistic_grid.fit(X_train_scaled, ycls_train)
logistic_pred = logistic_grid.best_estimator_.predict(X_test_scaled)

print("Best Ridge parameters:", ridge_grid.best_params_)
print("Best Ridge CV MSE:", -ridge_grid.best_score_)
print("Tuned Ridge test metrics:", regression_metrics(yreg_test, ridge_pred))
print("\nBest Logistic Regression parameters:", logistic_grid.best_params_)
print("Best Logistic CV accuracy:", logistic_grid.best_score_)
print("Tuned Logistic test metrics:", classification_metrics(ycls_test, logistic_pred))

ConfusionMatrixDisplay.from_predictions(ycls_test, logistic_pred, display_labels=["Low", "Good"], cmap="Blues")
plt.title("Tuned Logistic Regression – Confusion Matrix")
plt.show()
```

Best Ridge parameters: {'alpha': 10}
Best Ridge CV MSE: 0.4159121102632296
Tuned Ridge test metrics: {'MAE': 0.49808945937847876, 'MSE': 0.42225026351097095, 'RMSE': np.float64(0.6498078666121017), 'R2': 0.39180328310319634}
Best Logistic Regression parameters: {'C': 10}
Best Logistic CV accuracy: 0.7505254908532997
Tuned Logistic test metrics: {'Accuracy': 0.777292576419214, 'Precision': 0.8016528925619835, 'Recall': 0.782258064516129, 'F1': 0.7918367346938775}



Task 6 — Compare approaches

Compare the number of retained features, cross-validation mean and standard deviation, and test-set metrics. Do not assume one technique will always be best; the results depend on this dataset and the classification threshold.

```
reg_results = results_df[results_df["Task"] == "Regression"].sort_values("RMSE")
cls_results = results_df[results_df["Task"] == "Classification"].sort_values("F1", ascending=False)

print("Regression comparison (lower MAE/RMSE, higher R² are desirable):")
display(reg_results[["Feature set", "Feature count", "MAE", "RMSE", "R2", "CV mean", "CV std"]])

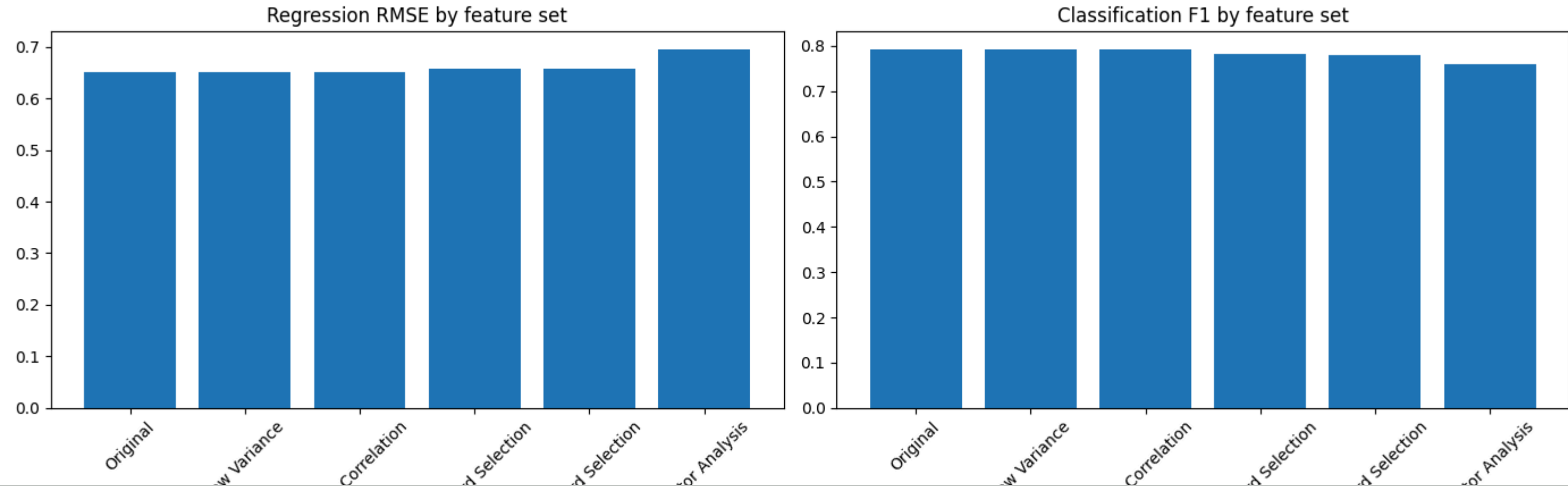
print("\nClassification comparison (consider precision, recall, and F1 together):")
display(cls_results[["Feature set", "Feature count", "Accuracy", "Precision", "Recall", "F1", "CV mean", "CV std"]])

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].bar(reg_results["Feature set"], reg_results["RMSE"])
axes[0].set_title("Regression RMSE by feature set")
axes[0].tick_params(axis="x", rotation=45)
axes[1].bar(cls_results["Feature set"], cls_results["F1"])
axes[1].set_title("Classification F1 by feature set")
axes[1].tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```

Regression comparison (lower MAE/RMSE, higher R² are desirable):							
Feature set	Feature count	MAE	RMSE	R²	CV mean	CV std	
0	Original	11	0.497891	0.650092	0.391272	0.343928	0.072757
1	Low Variance	11	0.497891	0.650092	0.391272	0.343928	0.072757
2	High Correlation	11	0.497891	0.650092	0.391272	0.343928	0.072757
3	Backward Selection	5	0.503251	0.656764	0.378712	0.349696	0.065464
4	Forward Selection	5	0.503251	0.656764	0.378712	0.349696	0.065464
10	Factor Analysis	5	0.539159	0.694813	0.304639	0.287771	0.093852

Classification comparison (consider precision, recall, and F1 together):

Feature set	Feature count	Accuracy	Precision	Recall	F1	CV mean	CV std
5	Original	11	0.777293	0.801653	0.782258	0.791837	0.750525
6	Low Variance	11	0.777293	0.801653	0.782258	0.791837	0.750525
7	High Correlation	11	0.777293	0.801653	0.782258	0.791837	0.750525
8	Backward Selection	5	0.768559	0.798319	0.766129	0.781893	0.761448
9	Forward Selection	5	0.768559	0.803419	0.758065	0.780083	0.738480
11	Factor Analysis	5	0.742358	0.768595	0.750000	0.759184	0.716532



Task 7 — Self-learning and observations

Use your actual outputs to complete the observations below.

- Low Variance Filter:** report whether any constant or near-constant features were removed.
- High Correlation Filter:** report the correlated features removed and discuss whether redundancy was reduced.
- Factor Analysis:** compare the number of latent factors with the original number of features; note that factors are combinations of variables.
- Backward and Forward Selection:** compare the selected feature subsets for regression and classification.
- Gradient Descent:** identify learning rates that gave smooth loss reduction. Very high learning rates may cause unstable updates.
- Cross-validation:** compare mean and standard deviation to assess average performance and variability.
- GridSearchCV:** report the selected Ridge alpha and Logistic Regression C.
- Improvement goal:** for regression, reduce MAE/RMSE and increase R²; for classification, evaluate accuracy alongside precision, recall, and F1. Do not claim a model improved unless the displayed results support it.

Important limitation

The dataset's `quality` score is an integer rating. It is treated as a numeric target for the required Linear Regression demonstration. For Logistic Regression, it is converted to a binary label using the stated threshold. These are two different tasks derived from the same source column.